# NICE – a nice 32 bit architecture

## An introduction for beginners

The nice_cpu project

The NICE architecture is by Bernd Ulmann

October 2026

# Goal

Why another 32 bit processor? Why not just use a commodity one?

- First of all, there is nothing like understanding a CPU down to the last bit – and then building it yourself.
- NICE ("NICE Is Charmingly Elegant") was designed by Bernd Ulmann and described in two articles, in 1997 [1] and 1999 [2], and on a web page with an emulator in 2004 [3].
- In 2006, its 16 bit descendant QNICE ("Quite NICE") followed [4]. QNICE runs on FPGAs today.
- This project gives NICE a complete specification, an assembler, an emulator, a runtime library, a WebAssembly translator, and a first FPGA core – and next, a faster one.
- This introduction explains NICE step by step. It assumes you know what a register and a memory address are, and nothing more.

# Basics

- 32 bit data and 32 bit addresses
- Memory is **word addressed**: every address holds one 32 bit word. There are no bytes.
- 16 registers of 32 bit, `R0...R15`
- One instruction word, followed by zero to three constant words
- Three operand instructions: `ADD R1, R2, R3` means `R1` := `R2` + `R3`
- Every operand can use every one of 8 addressing modes, so memory to memory operations are normal
- **Every** instruction is conditional
- Small instruction set: 16 ALU operations, 16 shifts, and 3 control instructions
- Flags change only when an instruction asks for it

# Registers

- A program sees 16 registers:

| R0 | R1 | ... | R12 | R13 | R14 | R15 |
|---|---|---|---|---|---|---|

- Most are general purpose, but four are special:

  R0: Always reads as 0. Writes to it are thrown away. This is handy for constants and comparisons.

  R13: The stack pointer (SP), by convention only. The hardware treats it like any other register.

  R14: The status register (SR).

  R15: The program counter (PC). Writing to it is a jump.

- Unlike QNICE, NICE has no register banks. A subroutine saves the registers it uses on the stack.

# The status register R14

The upper 8 bits hold the flags. The lower 24 bits (VBASE) say where the interrupt vector table is:

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23...0 |
|---|---|---|---|---|---|---|---|---|
| I | E | V | N | Z | C | X | 1 | VBASE |

- 1: Always 1. This makes "always" a condition.
- X: 1 if the last result was 0xFFFFFFFF
- C: Carry (or borrow) flag
- Z: 1 if the last result was 0x00000000
- N: 1 if the last result was negative (bit 31 set)
- V: 1 if the last operation caused a signed overflow
- E: If 1, maskable interrupts are enabled
- I: Set to 1 when an interrupt is taken

# Input/Output

- All input and output is memory mapped, so there are no special I/O instructions. A device is read and written like any memory word.
- The upper 256 words of memory, `0xFFFFFF00` to `0xFFFFFFFF`, are reserved for I/O devices.
- The emulator has a console at `0xFFFFFF00`. Each word written there prints one character, from bits 7..0:

  `MOVE @0xFFFFFF00, 0x41` prints `A`

- The FPGA implementation will have a UART at the same address.
- Reading a memory address where nothing is connected returns 0.

# Instruction set basics

- Almost every instruction has three operands: one destination and two sources, **destination first**:

  `SUB R1, R2, R3` means $\texttt{R1} := \texttt{R2} - \texttt{R3}$

  QNICE has it the other way round (`SUB R3, R1`), so take care.
- The fields are called `dest`, `s0`, and `s1`.
- Operations that need only one source, such as `MOVE` and `NOT`, take `dest` and `s0`.
- Two modifiers can follow the mnemonic:

  `[M]`: Modify the flags. Without it, the flags stay as they are. `SUB[M] R1, R2, R3` sets `Z` if $\texttt{R2} = \texttt{R3}$.

  `[C]`: Use the carry flag as carry in (for `ADD`, `SUB`, `MDBL`, and `DBL`).
- A condition can come first: `?Z ADD R1, R1, 1` adds 1 only if `Z` is set.

# Instruction format

- ALU and shift instructions share one 32 bit format:

| 31 | 30..28 | 27 | 26 | 25 | 24..21 | 20..14 | 13..7 | 6..0 |
|---|---|---|---|---|---|---|---|---|
| NEG | SEL | 0 | C | M | opcode | dest | s0 | s1 |
| NEG | SEL | 1 | 0 | M | opcode | dest | s0 | s1 |

  NEG and SEL are the condition. Bits 27..26 are the class: 0x ALU, 10 shift, 11 control.

- Each operand field is 7 bits, a mode and a register:

| 3 bit | 4 bit |
|---|---|
| mode | register |

- The control instructions use this format:

| 31 | 30..28 | 27 | 26 | 25..21 | 20..8 | 7..0 |
|---|---|---|---|---|---|---|
| NEG | SEL | 1 | 1 | command | 0 | vector |

# Conditional execution

- SEL picks one of the upper 8 bits of SR. The instruction runs only if that bit is 1, or with NEG = 1, only if it is 0:

| SEL | Flag | Condition | With NEG = 1 |
|---|---|---|---|
| 000 | 1 | ?1 or nothing (always) | ?!1 (never) |
| 001 | X | ?X | ?!X |
| 010 | C | ?C | ?!C |
| 011 | Z | ?Z | ?!Z |
| 100 | N | ?N | ?!N |
| 101 | V | ?V | ?!V |
| 110 | E | ?E | ?!E |
| 111 | I | ?I | ?!I |

- Because bit 24 of SR is always 1, SEL = 0 means “always”.
- A skipped instruction does nothing at all, but the processor still steps over its constant words.

# List of instructions: ALU

| Opc | Instr | Operands | Effect |
|---|---|---|---|
| 0 | MOVE | dest, s0 | dest := s0 |
| 1 | SUB | dest, s0, s1 | dest := s0 - s1 (- C) |
| 2 | MDBL | dest, s0, s1 | dest := s0 + (s0 AND s1) (+ C) |
| 3 | ADD | dest, s0, s1 | dest := s0 + s1 (+ C) |
| 4 | DBL | dest, s0 | dest := s0 + s0 (+ C) |
| 5 | DEC | dest, s0 | dest := s0 - 1 |
| 6 | NOT | dest, s0 | dest := NOT s0 |
| 7 | NOR | dest, s0, s1 | dest := NOT (s0 OR s1) |
| 8 | IAND | dest, s0, s1 | dest := (NOT s0) AND s1 |
| 9 | NAND | dest, s0, s1 | dest := NOT (s0 AND s1) |
| A | XOR | dest, s0, s1 | dest := s0 XOR s1 |
| B | IOR | dest, s0, s1 | dest := (NOT s0) OR s1 |
| C | XNOR | dest, s0, s1 | dest := NOT (s0 XOR s1) |
| D | AND | dest, s0, s1 | dest := s0 AND s1 |
| E | ONE | dest | dest := 1 |
| F | OR | dest, s0, s1 | dest := s0 OR s1 |

The carry in, (+ C) or (- C), is used only with [C].

# Some unusual operations

- The operations come from the 74181, a classic ALU chip. That explains a few odd ones:

  `MDBL`: "Masked double": doubles `s0` where `s1` has ones. With `s1` = `0xFFFFFFFF` it is $2 \cdot$ `s0`.

  `IAND, IOR`: AND and OR with `s0` inverted. `IAND R1, R2, R1` clears in `R1` the bits that are set in `R2`.

  `ONE`: Sets `dest` to 1.

- Some things are missing on purpose:
  - There is no `CMP`. Subtract into `R0` instead, which keeps only the flags (see "Comparisons").
  - There is no `JMP`. Write to `PC` instead.
  - There is no multiply or divide. The runtime library has routines for that, such as `__mul`.

# List of instructions: shifts

Shifts take the value in `s0` and the count in `s1`. The name shows where the new bit comes from and where the old bit goes:

| Opc | Instr | Meaning | Opc | Instr | Meaning |
|---|---|---|---|---|---|
| 0 | 0<0, SHL | shift in 0 | 8 | 0>0, SHR | shift in 0 |
| 1 | 0<1 | shift in 1 | 9 | 1>0 | shift in 1 |
| 2 | 0<C | shift in C | A | C>0 | shift in C |
| 3 | C<0 | in 0, out to C | B | 0>C | in 0, out to C |
| 4 | C<1 | in 1, out to C | C | 1>C | in 1, out to C |
| 5 | C<C | rotate through C | D | C>C | rotate through C |
| 6 | <<<, ROL | rotate | E | >>>, ROR | rotate |
| 7 | C<<, BPL | rotate, copy to C | F | >>C, BPR | rotate, copy to C |

- Left shifts are A<B: B goes into bit 0, and bit 31 goes to A (0 means it is lost).
- Right shifts are A>B: A goes into bit 31, and bit 0 goes to B.
- The bit goes to C only with [M].

# Shift examples

- Multiply `R1` by 16:

```
SHL R1, R1, 4
```

- Unsigned divide `R1` by 2, and keep the remainder in `C`:

```
0>C[M] R1, R1, 1
```

- Swap the two halves of `R1`:

```
ROL R1, R1, 16
```

- There is no arithmetic shift right, but two instructions do it. The first copies the sign bit into `C` and throws the result away in `R0`. The second shifts `C` in from the left:

```
C<0[M] R0, R1, 1
C>0    R1, R1, 3
```

- The count uses all 32 bits of `s1`. A shift (but not a rotation) by 32 or more shifts everything out.

# Control instructions

The control class has seven commands, chosen by bits 25..21:

| Cmd bits | Command | Word | Description |
|---|---|---|---|
| 00000 | HALT | 0x0C000000 | Halt the processor |
| 00001 | RTI | 0x0C200000 | Return from interrupt |
| 00010 | INT n | 0x0C400000 + n | Software interrupt with vector n (0–255) |
| 00100–00111 | RDSPC, RDSSR, WRSPC, WRSSR | 0x0C800000–0x0CE00000 + x | Read or write the saved PC or SR |

- All other commands are reserved. They halt the processor.
- Control instructions are conditional too: `?Z HALT` halts only if `Z` is set.
- They never change the flags `X`, `C`, `Z`, `N`, or `V`, except that `RTI` restores all of `SR`, and `RDSPC R14` and `RDSSR R14` write it.

# Affected condition bits

With `[M]`, an instruction (W)rites the flags, or sets them to (0). It (R)eads `C` when `C` is used as an input:

| Instruction | V | N | Z | C | X |
|---|---|---|---|---|---|
| `ADD`, `SUB`, `MDBL`, `DBL` | W | W | W | W, R with `[C]` | W |
| `DEC` | W | W | W | W | W |
| `MOVE`, `ONE`, and the logic operations | 0 | W | W | 0 | W |
| Shifts that send a bit to `C` | 0 | W | W | W | W |
| Other shifts | 0 | W | W | 0 | W |
| `HALT`, `INT` | | | | | |
| `RTI` | all restored | | | | |

- Without `[M]`, no flag changes.
- The shifts that shift in `C` read it, with or without `[M]`.
- `MOVE[M] R0, R5` is a handy test: `Z`, `N`, and `X` then describe `R5`.

# Comparisons

There is no `CMP`, but `SUB[M]` into `R0` does the same job. `SUB[M] R0, a, b` computes $a - b$, throws the result away, and sets the flags:

| Condition | `Z` | `C` | Test |
|---|---|---|---|
| $a = b$ | 1 | 0 | `?Z` |
| $a \neq b$ | 0 | | `?!Z` |
| $a < b$, unsigned | 0 | 1 | `?C` |
| $a \geq b$, unsigned | | 0 | `?!C` |

- For $a > b$, swap the operands: `SUB[M] R0, b, a`, then `?C`.
- Signed $a < b$ is `N` $\neq$ `V`. That takes more than one conditional instruction to test.

# Addressing modes: sources

Every operand can use one of eight addressing modes. For the sources `s0` and `s1`:

| Mode | Notation | Description |
|---|---|---|
| 000 | `Rx` | The contents of `Rx` |
| 001 | `--Rx` | Decrement `Rx`, then use it |
| 010 | `Rx++` | Use `Rx`, then increment it |
| 011 | `c[Rx]` | The number `Rx` + $c$ |
| 100 | `@Rx` | The memory word at address `Rx` |
| 101 | `@--Rx` | Decrement `Rx`, then use the memory word there |
| 110 | `@Rx++` | Use the memory word at `Rx`, then increment `Rx` |
| 111 | `@c[Rx]` | The memory word at address `Rx` + $c$ |

- The `@` means "in memory". Mode bit 2 says the same.
- `c` is a **constant word** that follows the instruction word (modes `011` and `111`).

# Addressing modes: destinations

The destination `dest` uses the same eight modes:

| Mode | Notation | Description |
|---|---|---|
| 000 | `Rx` | Write `Rx` |
| 001 | `Rx--` | Write `Rx`, then decrement it |
| 010 | `Rx++` | Write `Rx`, then increment it |
| 011 | `c[Rx]` | Same as `@c[Rx]` |
| 100 | `@Rx` | Write the memory word at address `Rx` |
| 101 | `@--Rx` | Decrement `Rx`, then write the memory word there |
| 110 | `@Rx++` | Write the memory word at `Rx`, then increment `Rx` |
| 111 | `@c[Rx]` | Write the memory word at address `Rx` + $c$ |

- The sources are read first (`s0`, then `s1`), and the destination is written last.
- The destination is never read, so `ADD @R1, @R1, 1` reads the memory word once and writes it once.

# Using constant operands

A constant is a word that follows the instruction. The assembler writes it for you:

- `c[R0]` is just `c`, because `R0` is always 0. So this is how you write an **immediate** value:

```
MOVE R1, 0x1234
```

- `@c[R0]` is `@c`, a fixed (**absolute**) memory address:

```
MOVE @0x40, R1
```

- With a register, it is an **indexed** address, for example the third word of a table that `R2` points to:

```
MOVE R1, @2[R2]
```

- One instruction can have up to three constants. They follow the instruction word in the order `dest`, `s0`, `s1`. `ADD @0x40, R1, 7` is three words long.

# Addressing mode examples

- Copy `R2` to `R1`:

```
MOVE R1, R2
```

- Store `R2` in the memory word that `R1` points to:

```
MOVE @R1, R2
```

- Using `R13` as the stack pointer, push `R1` onto the stack, and pop it again:

```
MOVE @--R13, R1      MOVE R1, @R13++
```

- Copy a word from one place in memory to another, and move both pointers on – all in one instruction:

```
MOVE @R2++, @R1++
```

- Add 1 to a counter in memory:

```
ADD @COUNT, @COUNT, 1
```

# Jumps

- `R15` is the program counter. A jump is just a write to `R15`. With `PC .EQU R15`:

```
MOVE PC, LOOP        Jump to LOOP
?!Z MOVE PC, LOOP    Jump if Z is not set
ADD PC, PC, 10       Jump forward by 10 words
```

- When an instruction reads `R15`, it gets the address of the **next** instruction, after all constant words.
- So `ADD PC, PC, 10` jumps to 10 words after the end of the `ADD`.
- Any write to `R15` jumps, even with other addressing modes. For example, `MOVE PC, @R13++` pops an address off the stack and jumps there.

# Subroutine calls

- There is no call instruction. A call is two instructions, with `SP .EQU R13`:

```
ADD  @--SP, 2, PC   Push the return address
MOVE PC, ROUTINE    Jump to the subroutine
```

- Why 2? `PC` reads as the address after the `ADD`, which is where the 2 word `MOVE` starts. The return address is after the `MOVE`.
- To return, pop the return address into the program counter:

```
MOVE PC, @SP++
```

- The stack grows down, and `SP` points at the last word pushed.
- By convention, arguments are passed in `R1`...`R4`, and the result comes back in `R1`. `R7`...`R13` must be the same after the call as before.

# Examples

The following examples show how instructions are encoded:

| NEG | SEL | class, C | M | opcode | dest | s0 | s1 | Hex |
|---|---|---|---|---|---|---|---|---|
| MOVE R1, R2 | | | | | | | | |
| 0 | 000 | 00 | 0 | 0000 | 0000001 | 0000010 | 0000000 | 0x00004100 |
| DEC[M] R2, R2 | | | | | | | | |
| 0 | 000 | 00 | 1 | 0101 | 0000010 | 0000010 | 0000000 | 0x02A08100 |
| ?!Z MOVE R15, 4 | | | | | | | | |
| 1 | 011 | 00 | 0 | 0000 | 0001111 | 0110000 | 0000000 | 0xB003D800 |
| | | | | | | constant word for s0 | | 0x00000004 |
| SHR R1, R1, 4 | | | | | | | | |
| 0 | 000 | 10 | 0 | 1000 | 0000001 | 0000001 | 0110000 | 0x090040B0 |
| | | | | | | | constant word for s1 | 0x00000004 |
| ADD @--R13, 2, R15 | | | | | | | | |
| 0 | 000 | 00 | 0 | 0011 | 1011101 | 0110000 | 0001111 | 0x0077580F |
| | | | | | | constant word for s0 | | 0x00000002 |

# A complete example

One instruction can do a lot. This one is from the 2004 NICE web page:

```
?!Z ADD[M] @R3++, --R2, @0x12345678[R1]
```

It is encoded as `0xB278C971`, followed by the constant `0x12345678`. If `Z` is 0, it does this:

1. Decrement `R2`, and use it as `s0`.
2. Read the memory word at `R1` + `0x12345678` as `s1`.
3. Store `s0` + `s1` at the address in `R3`, then increment `R3`.
4. Update the flags `X`, `C`, `Z`, `N`, and `V`.

If `Z` is 1, it does nothing, and the program continues after the constant word.

# Code example: $\sum_{i=1}^{0x1000} i$

The file `doc/intro/examples/sum.nasm`, with the address and words of each instruction:

```
                                PC      .EQU    R15
                                        .ORG    0
00000000 00005800 00000000              MOVE    R1, 0
00000002 00009800 00001000              MOVE    R2, 0x1000
00000004 00604082               LOOP    ADD     R1, R1, R2
00000005 02a08100                       DEC[M]  R2, R2
00000006 b003d800 00000004          ?!Z MOVE    PC, LOOP
00000008 0c000000                       HALT
```

- `R1` collects the sum, and `R2` counts down from `0x1000`.
- `DEC[M]` sets `Z` when `R2` reaches 0. Until then, `?!Z MOVE PC, LOOP` jumps back.
- At the end, `R1` $=$ `0x800800` $= 4096 \cdot 4097 \,/\, 2$.

# Code example: copy a block of memory

```
PC      .EQU    R15
        MOVE    R1, SRC         ; source pointer
        MOVE    R2, DST         ; destination pointer
        MOVE    R3, 5           ; word count
LOOP    MOVE    @R2++, @R1++    ; copy a word, advance both pointers
        DEC[M]  R3, R3
    ?!Z MOVE    PC, LOOP
        HALT

SRC     .DAT    1, 2, 3, 4, 5
DST     .BLK    5               ; 5 words of 0
```

- Labels are words at the start of a line. Their value is an address.
- `.DAT` puts words into memory, and `.BLK` reserves words that start as 0.
- Everything after `;` is a comment.

# Code example: Hello, NICE!

```
SP      .EQU    R13
PC      .EQU    R15
CONSOLE .EQU    0xFFFFFF00

        MOVE    SP, 0x1000      ; set up the stack
        MOVE    R1, TEXT        ; the argument: the string's address
        ADD     @--SP, 2, PC    ; push the return address ...
        MOVE    PC, PUTS        ; ... and jump: a subroutine call
        MOVE    @CONSOLE, 10    ; a newline
        HALT

; PUTS prints the string at R1. The string ends with a 0 word.
PUTS    MOVE[M] R2, @R1++       ; next character, and set Z if it is 0
    ?Z  MOVE    PC, @SP++       ; at the end: return
        MOVE    @CONSOLE, R2    ; print the character
        MOVE    PC, PUTS        ; and loop

TEXT    .ASCII_L 'Hello, NICE!'
```

`.ASCII_L` stores one character per word and adds a 0 word at the end. The conditional return `?Z MOVE PC, @SP++` needs no branch around it.

# Interrupts

NICE has a simple interrupt scheme, the same as QNICE's:

- There are three kinds of interrupt: a maskable request (*IRQ*), a non-maskable request (*NMI*), and the software interrupt `INT n`.
- IRQs are accepted only while the flag `E` is 1. After reset it is 0.
- When an interrupt is taken, the processor saves `PC` and `SR` in two hidden registers, sets `I`, and jumps through the vector table.
- The interrupt service routine (*ISR*) ends with `RTI`, which restores `PC` and `SR`. So the flags are safe, but the ISR must save any other register it uses.
- Interrupts cannot be nested. A request that arrives during an ISR waits until the `RTI`.

# The vector table

- The vector table has 256 words. Word $v$ holds the address of the ISR for vector $v$.
- The device that requests an interrupt supplies the vector number. `INT n` uses `n`.
- The table starts at address `VBASE` $\cdot$ 256, where `VBASE` is bits 23..0 of `SR`.
- So to put the table at `0x100` and then allow IRQs, with `SR .EQU R14`:

```
OR SR, SR, 1            VBASE = 1
OR SR, SR, 0x40000000   E = 1
```

- After reset, `PC` $= 0$ and `VBASE` $= 0$, so word 0 is both the first instruction and vector table entry 0. Set `VBASE` before you allow interrupts.

# Code example: an interrupt routine

```
SP      .EQU    R13
SR      .EQU    R14
PC      .EQU    R15
CONSOLE .EQU    0xFFFFFF00

        .ORG    0
        MOVE    SP, 0x1000
        OR      SR, SR, 1               ; VBASE = 1: the table is at 0x100
        OR      SR, SR, 0x40000000      ; E = 1: accept IRQs
        MOVE    R1, 20
WAIT    DEC[M]  R1, R1                  ; count down from 20 ...
    ?!Z MOVE    PC, WAIT                ; ... while the IRQs come in
        MOVE    @CONSOLE, 10            ; a newline
        HALT

ISR     MOVE    @CONSOLE, 0x2A          ; print '*'
        RTI                             ; restore PC and SR, and return

        .ORG    0x105                   ; vector 5 of the table at 0x100
        .DAT    ISR
```

`niceemu --irq 5:5 --irq 12:5 irq.bin` raises two IRQs with vector 5, and prints **.

# Soft- and hardware

Everything is in the repository `github.com/MJoergen/nice_cpu`:

- `doc/ISA.md`: The specification. When in doubt, it is the authority.
- `assembler/`: `niceasm`, the assembler, one Python 3 script.
- `emulator/`: `niceemu`, the emulator, written in Zig.
- `runtime/`: Start-up code and routines such as multiply, divide, and floating point.
- `translator/`: `nicewasm`, which translates WebAssembly into NICE assembly. That way, C, Rust, and Zig programs can run on NICE.
- `hdl/`: The FPGA implementation in VHDL: a multi-cycle core and a pipelined core, both checked against the emulator.

# Using the assembler and the emulator

Assemble the summation program and look at the result:

```
$ assembler/niceasm.py -o sum.bin doc/intro/examples/sum.nasm
$ cat sum.bin
00000000 00005800
00000001 00000000
00000002 00009800
00000003 00001000
...
```

Run it. The emulator prints the registers when it halts:

```
$ emulator/zig-out/bin/niceemu sum.bin
Halted (HALT) after 12291 steps, PC=00000009
R0  00000000 R1  00800800 R2  00000000 R3  00000000
R4  00000000 R5  00000000 R6  00000000 R7  00000000
R8  00000000 R9  00000000 R10 00000000 R11 00000000
R12 00000000 R13 00000000 R14 09000000 R15 00000009
```

R14 = 0x09000000: bit 24 is always set, and bit 27 is Z.

# Watching a program run

`--trace` prints one line per instruction: its address, its word, and what it changed. `-n` stops after a number of steps:

```
$ emulator/zig-out/bin/niceemu --trace -n 8 sum.bin
00000000 00005800
00000002 00009800 R2=00001000
00000004 00604082 R1=00001000
00000005 02a08100 R2=00000fff
00000006 b003d800 R15=00000004
00000004 00604082 R1=00001fff
00000005 02a08100 R2=00000ffe
00000006 b003d800 R15=00000004
Stopped at the step limit after 8 steps, PC=00000004
...
```

- The first line changes nothing: `R1` was 0 already.
- `R15` is shown only when an instruction jumps.
- The FPGA implementation will write the same trace, so the two can be compared line by line.

# Things to remember

- The destination comes first: `ADD dest, s0, s1`.
- The flags change only with `[M]`.
- Memory has words, not bytes. `.ASCII_L` stores one character per word.
- `R0` is always 0, and `R15` reads as the address of the next instruction.
- A comparison is `SUB[M] R0, a, b`, a jump is a write to `PC`, and a call is `ADD @--SP, 2, PC` followed by `MOVE PC, label`.
- Every instruction can be conditional, so short `if`s need no jumps:

```
SUB[M]  R0, R1, R2
?C  MOVE R1, R2     ; R1 := max(R1, R2), unsigned
```

# Future plans

- As of now (OCT-2026) the following exists:
  - a complete specification of the instruction set,
  - an assembler and an emulator,
  - a runtime library,
  - a WebAssembly translator, so that C, Rust, and Zig programs can run on the emulator, and
  - two FPGA cores, a simple multi-cycle one and a pipelined one, which write the same trace as the emulator for every test program.
- Next comes a faster clock for the **pipelined core**. The system around it, with RAM and a UART, is built in the separate NiceMonMEGA65 repository.
- A C compiler that targets NICE directly would be great, too. The calling convention for one is already in `runtime/ABI.md`.

# Bibliography



Bernd Ulmann, *NICE – an elegant and powerful 32-bit architecture*, Computer Architecture News 25(4), SEP-1997.


Bernd Ulmann, *Designing a NICE processor*, Microprocessors and Microsystems 23, 1999.


Bernd Ulmann, *The NICE Processor Pages*, `http://www.vaxman.de/projects/nice/nice.html`, 2004.


Bernd Ulmann, *QNICE – a nice 16 bit architecture*, `https://github.com/sy2002/QNICE-FPGA`.


*NICE instruction set architecture*, `doc/ISA.md` in `https://github.com/MJoergen/nice_cpu`.